HOW-code：基于硬冻结安全审查的大模型工具自举与进化协议

HOW-code: A Protocol for LLM Tool Self-Bootstrapping and Evolution via Hard-Freeze Safety Review

Name: Constant-Dalink
Location: Hubei, China
Contact: 13581432693@163.com / 13581432693
Independent Researcher

Preprint v1.0 — 2026

摘要

当前大模型智能体通常依赖预定义工具集运行。当任务超出现有工具边界时，系统往往只能失败，或转而让模型直接生成并执行代码。前者缺乏长期适应能力，后者则容易突破执行边界并带来安全风险。

本文提出 **HOW-code** 协议，用于在“能力扩展”与“安全控制”之间建立明确机制。其基本做法是，将智能体划分为两个角色：**执行者模型（Executor）**与 **创造者模型（Creator）**。执行者负责在现有工具集内完成任务；创造者仅在检测到能力缺口时生成新工具代码。新工具生成后，系统必须进入强制性的 **硬冻结闸门（Hard Freeze Gate）**：执行流被同步阻断，直到人类完成审查并给出批准或拒绝结论。只有通过审查的工具，才会被注册到工具库中，并进入未来任务的可用能力集合。

本文的主要工作包括：(1) 给出 HOW-code 的双角色架构，将运行期执行与能力扩展分离；(2) 使用形式化状态机定义从 `hc <name>` 到代码生成、冻结审查与工具注册的完整协议；(3) 将 Constant-Dalink 不完备公式引入工具进化问题，说明按需造器优于预载全部潜在工具；(4) 说明 Hard Freeze Gate 在能力扩张场景下相较异步式人工干预具有更强的同步约束；(5) 给出 HOW-code 在 GSS-OS 中的工程映射：`howcode.py` → `gate.py` → `lib_manager.py`。

本文的目标不是论证系统应当完全自治，而是给出一种在能力可增长的前提下仍保持边界清晰的协议结构。

关键词：LLM Agent；工具自举；工具进化；Human-in-the-Loop；Hard Freeze；安全审查；代码生成；动态工具库

1. 引言

1.1. 问题背景

现有 Agent 系统在能力来源上大致可分为两类：

1. **静态工具型**：开发者预先定义全部工具，模型只负责选择与调用；
2. **代码生成型**：模型在任务过程中直接生成脚本或代码，并交由系统执行。

前者的问题在于能力边界固定。一旦任务超出现有工具集，系统缺少扩展路径。

后者的问题在于执行边界不清晰。如果模型被允许直接生成并运行代码，那么生成错误、越权调用、工具污染以及不可预期副作用都会成为实际风险。

因此，一个可持续的智能体系统需要同时满足两个条件：

- **可扩展性**：在工具不足时，能够生成新工具；
- **可控性**：新工具不能在未经审查的情况下进入执行面。

HOW-code 的目标，就是用一套明确协议同时满足这两个条件。

1.2. 基本观点

本文的基本观点是：

> 工具生成不应被视为普通动作调用，而应被视为对系统能力边界的修改。

因此，它不应被处理为普通异步任务。一旦系统请求生成新工具，就应进入一种明确的模式切换：当前执行流暂停，待生成代码经过审查后，再决定是否将其注册为新能力。

本文将这一机制记为：

Hard Freeze Gate

1.3. 与前两篇工作的关系

本文与前两篇工作在理论上保持一致：

- 从图灵完备性角度看，代码生成提供了系统的外部扩展能力；
- 从 Constant-Dalink 不完备性角度看，系统不应预载所有未来可能工具，而应在能力缺口出现时按需生成。

因此，HOW-code 讨论的不是任意代码生成，而是在最小上下文原则和显式审查机制下的能力扩展协议。

2. 相关工作

2.1. 工具调用与工具学习

Toolformer [1]、函数调用框架以及多种 Agent 系统表明，大模型可以学会在合适时机调用外部工具。但这些工作通常默认工具集已经存在，主要讨论的是“何时调用工具”，而不是“工具集如何扩展”。

2.2. 代码生成型智能体

另一类系统允许模型在任务过程中生成 Python、Shell 或 JavaScript 代码并直接执行。这类方法的优点是灵活，但也存在几个共同问题：

- 代码生成与代码执行之间缺乏明确边界；
- 人工审查常常是可选项，而不是协议中的强制步骤；
- 审查机制经常以异步通知、日志回看或后验确认的方式实现。

这些做法适用于一般任务辅助，但对于“能力扩展”这一问题，仍缺少更严格的控制结构。

2.3. 人工在环机制

已有许多系统引入 Human-in-the-Loop。但在多数实现中，人工确认具有以下特征：

- 异步弹窗；
- 允许超时；
- 后台等待；
- 审查期间系统仍可继续运行。

对于普通动作确认，这样的机制可能足够。但对于新工具注册而言，问题不只是“是否执行某一步”，而是“是否扩大未来所有任务的可执行能力边界”。这一点要求更强的同步约束。

3. Constant-Dalink 不完备公式与按需造器原理

3.1. 公式重述

本文继续使用 Constant-Dalink 不完备公式作为认知压缩与能力演化的基础：

$$\forall \pm \{(V_1, V_2)^c, V_1, V_2, V, \dots\} \xrightarrow{\text{Check}} \circ V_{use}$$

其含义是：在任意正负加载状态下，系统在当前任务中真正需要激活的知识，始终只是知识空间中的极小子集 V_{use}。

3.2. 对工具系统的解释

设：

- $\mathcal{K}$：全部可能工具的宇宙；
- $\Lambda_t \subset \mathcal{K}$：时刻 t 已注册并可加载的工具集；
- $\mathcal{O}_{core}$：GSS 的核心原子指令集；
- τ：当前任务。

定义当前系统的能力闭包为：

$$\mathcal{C}_t = \mathrm{Cl}(\mathcal{O}_{core} \cup \Lambda_t)$$

其中 Cl 表示在核心指令与已知工具下可组合得到的能力集合。

记任务所需能力为：

$$\mathcal{R}(\tau)$$

若：

$$\mathcal{R}(\tau) \subseteq \mathcal{C}_t$$

则系统无需扩展，可以直接执行。若：

$$\Delta_t(\tau) = \mathcal{R}(\tau) \setminus \mathcal{C}_t \neq \varnothing$$

则称出现了**能力缺口**。

3.3. 按需造器的必要性

一种常见思路是预先准备大量工具，以减少未来缺口。但根据 Constant-Dalink 公式，这种做法通常与认知经济性不一致：

$$|V_{use}(\tau)| \ll |\mathcal{K}|$$

即使工具宇宙很大，单个任务实际需要激活的工具和知识仍然很少。因此，更符合该公式的策略是：

1. 使当前上下文与已加载工具保持较小规模；

2. 仅在检测到能力缺口时触发工具生成；
3. 仅将通过审查的新工具写入长期工具库。

HOW-code 的设计正是基于这一原则。

4. 双角色架构：执行者模型与创造者模型

4.1. 角色定义

HOW-code 使用两个逻辑上分离的模型：

- **执行者模型** M_E：负责将任务 τ 映射为 GSS 指令序列；
- **创造者模型** M_C：负责在出现能力缺口时，根据工具规格生成代码工件。

形式化地：

$$M_E : (\tau, \Lambda_t, \mathcal{O}_{core}) \to \Pi$$

其中 Π 为 GSS 程序。

$$M_C : \rho \to z$$

其中 ρ 为工具规格，z 为代码工件。

4.2. 角色分离的原因

若由同一模型同时承担执行与造器角色，通常会出现三个问题：

1. **目标混杂**：当前任务完成与未来能力扩展处于同一生成上下文；
2. **上下文干扰**：执行期信息过多，不利于代码生成聚焦；
3. **责任不清**：难以区分错误来源于动作规划还是工具创建。

因此，HOW-code 要求将执行层与进化层分离，如表 1 所示。

表 1: 执行层与扩展层的角色分离

层级	角色	目标
运行层	执行者模型	完成当前任务
扩展层	创造者模型	生成未来可复用工具

4.3. 反馈结构

两类模型之间存在明确反馈关系：

$$\text{Task} \rightarrow \text{Gap} \rightarrow \text{Create} \rightarrow \text{Review} \rightarrow \text{Library} \rightarrow \text{Future Task}$$

这一过程与普通工具调用不同。它描述的不是一次单步执行，而是系统能力边界的更新流程。

5. HOW-code 协议的形式化定义

5.1. hc 指令

在 GSS 中，定义创造请求指令：

```
<hc_cmd> ::= "hc" SPACE <tool_name>
```

例如：

```
hc img_zip
```

其语义不是立即执行某个工具，而是：

请求系统为指定缺失能力生成工具提案。

5.2. 协议状态机

定义 HOW-code 的状态集：

$$Q = \{q_{exec}, q_{gap}, q_{create}, q_{freeze}, q_{review}, q_{commit}, q_{abort}, q_{resume}\}$$

各状态含义如表 2 所示。

表 2: HOW-code 协议状态

状态	含义
q_{exec}	正常执行
q_{gap}	检测到能力缺口
q_{create}	调用创造者模型生成代码
q_{freeze}	系统进入硬冻结
q_{review}	人类审查代码
q_{commit}	注册新工具
q_{abort}	拒绝生成结果
q_{resume}	返回执行流

状态转移函数定义为：

$$\delta : Q \times \Sigma \to Q$$

关键转移如下：

$$q_{exec} \xrightarrow{hc} q_{gap}$$

$$q_{gap} \xrightarrow{spec} q_{create}$$

$$q_{create} \xrightarrow{z} q_{freeze}$$

$$q_{freeze} \xrightarrow{\text{human}} q_{review}$$

$$q_{review} \xrightarrow{\text{approve}} q_{commit}, \qquad q_{review} \xrightarrow{\text{reject}} q_{abort}$$

$$q_{commit} \to q_{resume}, \qquad q_{abort} \to q_{resume}$$

5.3. 工具规格生成

给定任务 τ 与能力缺口 $\Delta_t(\tau)$，系统生成工具规格：

$$\rho = \mathrm{Spec}(\tau, \Delta_t, \Lambda_t)$$

规格通常包括：

- 工具名；
- 输入输出约束；
- 所属库；
- 可依赖 API；
- 安全限制；
- 不允许行为列表。

5.4. 代码生成与注册规则

创造者模型输出代码工件：

$$z = M_C(\rho)$$

定义审查函数：

$$r(z) \in \{0, 1\}$$

其中：

$$r(z) = 1 \Rightarrow \text{批准}, \qquad r(z) = 0 \Rightarrow \text{拒绝}$$

工具库更新规则定义为：

$$\Lambda_{t+1} = \begin{cases} \Lambda_t \cup \{h\}, & r(z) = 1, \\ \Lambda_t, & r(z) = 0. \end{cases}$$

其中 h 为由代码工件 z 对应的新工具实体。

6. Hard Freeze Gate：硬冻结的必要性

6.1. 定义

定义硬冻结闸门 G_H：

$$G_H(t_f) : \forall t \in [t_f, t_v),\ \mathrm{Act}(t) = \varnothing$$

其中：

- t_f：冻结开始时刻；
- t_v：审查结论给出时刻；
- $\mathrm{Act}(t)$：系统在时刻 t 产生的副作用动作集合。

该定义表示：

> 从冻结开始到人工给出明确结论之前，系统不得产生新的副作用。

6.2. 工程语义

在 GSS-OS 中，`gate.py` 不只是提示层弹窗，而是同步阻断机制。其实现语义可以概括为：

- 主执行流阻塞；
- 无自动超时释放；
- 审查结论必须显式给出；
- 未获结论前不得继续执行。

6.3. 为什么异步确认不够

若人工确认采用异步模式，则在等待期间仍可能发生：

1. 执行流继续推进；
2. 上下文状态变化；
3. 新代码被间接依赖；
4. 计划链条产生不可回滚副作用；
5. 审查对象与实际注册时的环境不一致。

因此，异步人工确认可以作为提醒机制，但不足以构成能力扩展阶段的安全边界。

7. 安全性定理

7.1. 未审查代码不可执行

定理 7.1 (Review-before-Execution). *若系统满足以下条件：*

1. *所有新工具只能由 `hc` 触发；*
2. *`hc` 后必经 q_{freeze}；*
3. *从 q_{freeze} 到 q_{commit} 的唯一出口是人工批准；*
4. *未经 q_{commit} 的工具不得加入 Λ；*

则任意代码工件 z，若其未获批准，则不可能进入可执行能力闭包。即：

$$z \notin \Lambda_{t+1} \setminus \Lambda_t \quad unless \quad r(z) = 1$$

证明要点：根据状态机定义，z 进入工具库仅存在如下路径：

$$q_{create} \to q_{freeze} \to q_{review} \xrightarrow{\text{approve}} q_{commit}$$

若无 `approve`，则无 `commit`，从而无工具注册。□

7.2. 硬冻结的线性化

定理 7.2 (Evolution Linearizability). *在 Hard Freeze 语义下，工具生成与工具注册构成一个可线性化事务。审查期间不存在并发副作用，因此新能力进入系统的时刻可视为单点提交事件。*

若冻结区间为 $[t_f, t_v)$，则：

$$\forall t \in [t_f, t_v),\ \mathrm{Act}(t) = \varnothing$$

因此，工具注册时刻 $t_c \geq t_v$ 可定义为系统能力边界的唯一跃迁点。这保证了扩展事件与执行事件之间的顺序关系清晰且可审计。□

7.3. 异步人工审查的不充分性

定理 7.3 (Async Review Insufficiency). *若人工审查采用异步模式，且系统在审查期间允许继续执行，则存在执行轨迹使得：*

1. *审查前后的环境状态不同；*
2. *工具提案对应的语境与提交时语境不一致；*
3. *因而无法保证能力扩张过程的事务一致性。*

证明思路：设工具提案在状态 x_t 下生成，但系统在等待审查期间继续运行至状态 x_{t+k}。若 $x_t \neq x_{t+k}$，则对 z 的批准对应的是旧状态下的提案，而不是当前状态下的能力注入。因此，审查与注册之间不具备串行一致性。□

这一结果说明：在能力扩张场景中，异步审查与硬冻结并不是强弱不同的同类机制，而是两种不同的控制结构。

8. HOW-code 在 GSS-OS 中的工程映射

8.1. `howcode.py`

`howcode.py` 是协议协调器，其主流程可概括为：

1. 接收 `hc <name>`；
2. 构造工具规格；
3. 调用 `ai_client.ask_creator()`；
4. 获取代码输出；

5. 调用 `gate.review()` 进入冻结审查；
6. 若通过，则调用 `lib_manager.register_tool()` 完成注册；
7. 返回执行层。

8.2. `gate.py`

`gate.py` 是协议中的核心控制模块。其主要要求为：

- 使用全局同步闸门；
- 冻结期间阻断执行流；
- 不设置自动超时放行；
- 审查结果必须由人类显式给出。

8.3. `lib_manager.py`

`lib_manager.py` 负责将批准后的工具转化为持久化资产，包括：

- 写入 `tools/`；
- 更新 `libs/*.yaml`；
- 支持 `mk_lib` 与 `mv`；
- 使未来的 `ld <lib>` 能加载新工具。

8.4. 与 Constant-Dalink 公式的对应

在 HOW-code 中，Constant-Dalink 公式可以具体写为：

$$\forall \pm \{(V_1, V_2)^c, V_1, V_2, V, \dots\} \xrightarrow{\text{Check}_\tau} \circ V_{use}(\tau)$$

若：

$$V_{use}(\tau) \subseteq \mathcal{C}_t$$

则系统直接执行。若：

$$V_{use}(\tau) \not\subseteq \mathcal{C}_t$$

则触发造器生成新能力项 V_{new}，并在人工批准后得到：

$$\mathcal{C}_{t+1} = \text{Cl}(\mathcal{O}_{core} \cup \Lambda_t \cup \{V_{new}\})$$

因此，HOW-code 不是对 Constant-Dalink 公式的例外，而是该公式在能力缺口场景中的扩展实现。

9. 示例：从能力缺口到工具沉淀

设用户任务为：

“生成一个图片压缩工具，并放入图片库。”

执行者模型检测到现有库中不存在压图能力，于是输出：

```
hc img_zip
mk_lib pic
mv img_zip pic
```

系统随后执行如下流程：

1. `hc img_zip`
   - 构造工具规格：输入图片路径、输出压缩结果、限制文件系统访问范围；
   - 调用创造者模型生成 `img_zip.py`。
2. 进入 `gate.review()`
   - 主执行流冻结；
   - 人类检查代码是否越权、是否满足预期、是否存在不安全行为。
3. 若批准
   - 执行 `register_tool(img_zip)`；
   - 执行 `mk_lib pic`；
   - 执行 `mv img_zip pic`。
4. 后续再遇到同类任务时
   - 执行者模型无需再次造器，只需加载相应库并调用工具。

这一示例说明：当前任务中的能力缺口，可以在审查通过后转化为未来任务中的稳定能力。

10. 与其他方案的比较

10.1. 与预载大量工具的比较

表 3: 预载大量工具与 HOW-code 的比较

方案	优点	局限
预载大量工具	初始覆盖面较广	上下文膨胀、检索混乱、维护成本高
HOW-code	按需扩展、长期沉淀	首次缺口需要人工审查

10.2. 与生成代码后立即执行的比较

表 4: 代码即生成即执行与 HOW-code 的比较

方案	灵活性	安全边界
代码即生成即执行	高	弱
HOW-code	高	明确且可审计

10.3. 与异步人工确认的比较

表 5: 异步人工确认与 Hard Freeze 的比较

方案	是否阻断执行	是否具备事务一致性
异步人工确认	否	否
Hard Freeze	是	是

11. 局限性与结论

11.1. 局限性

本文仍有若干局限：

1. **人工审查成本**：Hard Freeze 的代价是明显的。系统在能力扩张阶段需要暂停，审查过程会降低自动化流畅性。
2. **创造者模型质量依赖**：如果创造者模型的代码质量较低，则人工审查负担会增加。
3. **工具库治理问题**：当工具库持续增长后，还需要处理命名冲突、版本管理、依赖变更与废弃工具清理。

这些问题属于进一步工程治理范围，不影响本文协议本身的结构成立。

11.2. 结论

本文提出了 HOW-code，一种面向大模型智能体的工具自举与进化协议。

核心结论可概括如下：

表 6: 本文主要结论

#	内容	结论
1	双角色架构	执行者模型与创造者模型应逻辑分离
2	缺口触发机制	仅在能力缺口出现时触发造器
3	Constant-Dalink 融合	不预载全部未来工具，只激活最小可用能力集
4	Hard Freeze Gate	工具生成必须进入同步冻结审查
5	安全性定理	Hard Freeze 在能力扩张场景下相较异步人工干预具有更强的同步约束

本文的中心观点是：具备工具生成能力的智能体并不必然不安全，问题在于新工具是否可以在未经审查的情况下直接进入执行面。

因此，一个可扩展的 Agent 系统不应仅追求生成能力，还需要具备明确的能力边界修改协议。HOW-code 提供的就是这样一种结构：

- 执行阶段保持最小能力集；
- 发现缺口时按需生成；
- 生成后立即进入冻结审查；
- 审查通过后再注册为长期能力。

这一协议的目标不是削弱系统扩展能力，而是为能力扩展提供清晰、可审计、可控制的过程。

参考文献

[1] Timo Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS*, 2023.

[2] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.

[3] Andy Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. 2023.

[4] OpenAI. Function Calling and Tool Use Documentation. 2023–2024.

[5] Anthropic. Computer Use and Agentic Interaction Notes. 2024.

[6] Claude E. Shannon. A Mathematical Theory of Communication. 1948.

[7] GSS-OS Internal Notes. Constant-Dalink Incompleteness Formula and Cognitive Compression. 2025.

[8] GSS-OS Internal Notes. Hard Freeze Gate and Safe Tool Evolution Protocol. 2025.

A. HOW-code 协议伪代码

```
Input: task  , loaded libs Λt
Π ← Executor( , Λt)

for cmd in Parse(Π):
    if cmd.op != "hc":
        Execute(cmd)
    else:
          ← BuildSpec( , cmd.name, Λt)
        z ← Creator( )
        verdict ← HardFreezeReview(z)
        if verdict == approve:
            RegisterTool(z)
        else:
            Discard(z)
```

B. 核心不变量

定义以下安全不变量：

1. **I1: No-Unreviewed-Commit** 未审查代码不得入库。
2. **I2: No-Side-Effect-During-Freeze** 冻结区间内不得产生新副作用。
3. **I3: Reviewed-Provenance** 每个新工具必须对应明确的审查记录。
4. **I4: Minimal-Evolution Context** 创造者模型只接收必要规格，而非全部运行历史。